Specification: lta_cation_maxsym.py - MS2 occupation optimizer

2026-07-21

Contents

1. Purpose and stage boundary **2**

2. Dependencies and algorithmic provenance **3**

3. Mathematical model **3**
3.1 Parent action . . . 3
3.2 Occupation and stabilizer . . . 4
3.3 Orbit-coloring equivalence . . . 4

4. Population constraint resolution **4**
4.1 State and ring order . . . 4
4.2 Exact counts . . . 5
4.3 Ratio inputs . . . 5
4.4 Resolved records . . . 5

5. Occupation data structures **5**
5.1 `SiteOccupation` . . . 5
5.2 `OrbitColoringSolution` . . . 6
5.3 Verified solution . . . 6

6. Canonicalization and direct stabilizer verification **6**
6.1 Signature transformation . . . 6
6.2 Canonical signature . . . 6
6.3 Actual stabilizer . . . 7
6.4 Orbit-stabilizer invariant . . . 7

7. Exact orbit-coloring solver **7**
7.1 Deterministic orbit order . . . 7
7.2 Dynamic-programming state . . . 7
7.3 Fast rejection . . . 7
7.4 Search-node guard . . . 8

8. Maximum-order subgroup search **8**
8.1 Search domain . . . 8
8.2 Descending-order proof . . . 8
8.3 Actual stabilizer consistency . . . 8

9. Tie enumeration and ranking **9**
9.1 `first` mode . . . 9
9.2 `all` mode . . . 9
9.3 Deterministic tie ordering . . . 9

10. Optimality certificate **9**

11. Main public API **10**
11.1 Constraint resolution 10
11.2 Orbit coloring 10
11.3 Maximum-symmetry occupation optimizer 10

12. Failure semantics **11**

13. Complexity **11**

14. Verification requirements **11**

15. MS3 handoff **12**

18. MS7 hybrid occupation search **12**

Current authority routing: supported Group-I/II species, exact integer populations, formal charges, and charge neutrality are owned by `doc/specs/v7_13_group_i_ii_cation_extension.md`; the Al24Si24O96 charge-equivalent dry-volume model and empirical endpoint provenance are owned by `doc/specs/v7_14_group_i_ii_volume_reference_extension.md`. This document imports those semantics and does not redefine them.

1. Purpose and stage boundary

`lta_cation_maxsym.py` implements **MS2**, the maximum-symmetry optimizer for grouped-site occupations.

The module assigns one state to every grouped physical LTA cation site. With $\mathcal{C}$ denoting the full supported Group-I/II cation set,

$$\mathcal{C} = \{\mathrm{H}, \mathrm{Li}, \mathrm{Na}, \mathrm{K}, \mathrm{Rb}, \mathrm{Cs}, \mathrm{Fr}, \mathrm{Be}, \mathrm{Mg}, \mathrm{Ca}, \mathrm{Sr}, \mathrm{Ba}, \mathrm{Ra}\}, \qquad \mathcal{A} = \{\emptyset\} \cup \mathcal{C}.$$

For exact integer Group I/II cation counts within each 4R, 6R, and 8R family, it finds occupations whose stabilizer in the accepted fixed-cell parent group has maximum order.

MS2 implements:

- exact count and ratio resolution;
- vacancy inference;
- subgroup-orbit coloring;
- integer dynamic-programming feasibility;
- parent-orbit canonicalization;
- direct occupation-stabilizer verification;
- orbit-stabilizer validation;
- policy-correct subgroup conjugacy reduction;
- deterministic first-result and bounded all-ties modes;
- an explicit optimality certificate;
- hard failure when subgroup or coloring search cannot be completed.

MS2 does **not**:

- select off-center positional variants;
- optimize coordinate symmetry;
- construct `PlacedIon` records;
- enforce framework charge neutrality;
- rescale the lattice;
- write CIF, POSCAR, or LAMMPS files;
- perform density smoothing.

Exact static-coordinate realization begins in MS3. Structure construction and serialization remain in the existing population module and are integrated in MS4.

2. Dependencies and algorithmic provenance

Required package modules:

`lta_cation_template.py`
`lta_cation_symmetry.py`
`lta_cation_subgroups.py`

Runtime numerical dependencies are inherited through MS0. MS2 itself operates only on integer counts, finite-group tables, and permutation actions.

The general use of finite parent-group actions to remove symmetry-equivalent occupational configurations follows:

1. G. L. W. Hart and R. W. Forcade, "Algorithm for generating derivative structures," *Physical Review B* **77**, 224115 (2008), DOI: 10.1103/PhysRevB.77.224115.
2. R. Grau-Crespo, S. Hamad, C. R. A. Catlow, and N. H. de Leeuw, "Symmetry-adapted configurational modelling of fractional site occupancy in solids," *Journal of Physics: Condensed Matter* **19**, 256201 (2007), DOI: 10.1088/0953-8984/19/25/256201.
3. J.-C. Lian *et al.*, "Algorithm for generating irreducible site-occupancy configurations," *Physical Review B* **102**, 134209 (2020), DOI: 10.1103/PhysRevB.102.134209.

The decomposition of a parent site set into subgroup orbits follows the crystallographic group-subgroup viewpoint of:

E. Kroumova, J. M. Perez-Mato, and M. I. Aroyo, "WYCKSPLIT," *Journal of Applied Crystallography* **31**, 646 (1998).

The following are package-specific algorithms derived from standard group-action principles:

- maximizing stabilizer order instead of enumerating every occupation;
- exact orbit-count dynamic programming;
- explicit state ordering for canonical representatives;
- bounded tie enumeration;
- the optimality certificate and incomplete-search failure contract.

3. Mathematical model

3.1 Parent action

Let

$$G = G_{\text{site}}$$

be the complete accepted fixed-cell affine-operation group from MS0. Its physical-site action is

$$\phi_P : G \to \mathrm{Sym}(\mathcal{P}),$$

where $\mathcal{P}$ is the deterministic tuple of grouped physical sites in `LTATemplateModel`.

The action convention is

$$p_g[i] = j \quad \Longleftrightarrow \quad g(s_i) = s_j.$$

Each accepted operation must preserve ring family:

$$r(s_i) = r(p_g[i]).$$

MS2 rejects an action that maps between 4R, 6R, and 8R families.

3.2 Occupation and stabilizer

An occupation is a coloring

$$c : \mathcal{P} \to \mathcal{A}.$$

Its stabilizer is

$$H_{\mathrm{occ}}(c) = \{g \in G : c(gs) = c(s)\ \forall s \in \mathcal{P}\}.$$

MS2 solves

$$\boxed{\max_{c} |H_{\mathrm{occ}}(c)|}$$

subject to exact ring-family populations.

Chemical states are distinct. A symmetry operation may permute sites, but it may not exchange any supported cation species or vacancy labels.

3.3 Orbit-coloring equivalence

For a subgroup $H \leq G$, the physical sites split into orbits

$$\mathcal{P} = O_1 \sqcup \cdots \sqcup O_m.$$

An occupation preserves H if and only if it is constant on every H-orbit. Therefore each orbit receives one state in $\mathcal{A}$.

For orbit O_k and state α, define $x_{k\alpha} \in \{0, 1\}$. The exact constraints are

$$\sum_{\alpha \in \mathcal{A}} x_{k\alpha} = 1,$$

and, for each ring family r and state α,

$$\sum_{O_k \subseteq \mathcal{P}_r} |O_k| x_{k\alpha} = N_{r,\alpha}.$$

4. Population constraint resolution

4.1 State and ring order

The canonical state order is

```
empty < H < Li < Na < K < Rb < Cs < Fr < Be < Mg < Ca < Sr < Ba < Ra
```

and is represented by

```
OCCUPATION_STATES = ("empty", *SUPPORTED_CATIONS)
```

The preferred ring order is

```
4R, 6R, 8R
```

with any future ring labels appended lexicographically.

4.2 Exact counts

`resolve_population_constraints(..., counts_by_ring=...)` accepts nonnegative integer counts for every species in `SUPPORTED_CATIONS`. Vacancy is inferred as

$$N_{r,\emptyset} = |\mathcal{P}_r| - \sum_{\alpha \in \mathcal{C}} N_{r,\alpha}.$$

An explicit `empty` count is allowed only when it equals the inferred value.

A missing ring-family mapping means that the entire family is vacant. Counts above capacity, negative counts, Boolean values, unknown states, and unknown ring families are rejected.

4.3 Ratio inputs

`resolve_population_constraints(..., ratios_by_ring=...)` preserves the historical CLI semantics. For each supported cation state independently,

$$N_{r,\alpha} = \mathrm{round}(f_{r,\alpha}|\mathcal{P}_r|),$$

using Python's `round`. Vacancy is the remaining capacity.

Every fraction must lie in $[0, 1]$. If independently rounded cation counts exceed capacity, resolution fails. MS2 records the rounding rule in the resolved constraint object.

4.4 Resolved records

```python
@dataclass(frozen=True)
class RingPopulationConstraint:
    ring_class: str
    capacity: int
    counts: tuple[int, int, int, int]

@dataclass(frozen=True)
class ResolvedPopulationConstraints:
    site_ring_classes: tuple[str, ...]
    by_ring: tuple[RingPopulationConstraint, ...]
    source_kind: str
    rounding_rule: str
```

`counts` follows `OCCUPATION_STATES` exactly. For every ring family,

$$\sum_{\alpha \in \mathcal{A}} N_{r,\alpha} = |\mathcal{P}_r|.$$

5. Occupation data structures

5.1 `SiteOccupation`

```python
@dataclass(frozen=True)
class SiteOccupation:
    site_ids: tuple[str, ...]
    ring_classes: tuple[str, ...]
    state_by_site: tuple[str, ...]
```

Invariants:

- all tuples have identical nonzero length;

- `site_ids` are unique;
- each state belongs to `OCCUPATION_STATES`;
- ordering matches `LTATemplateModel.physical_sites`.

`counts_by_ring()` returns exact state counts. `occupied_site_count` excludes vacancy.

5.2 `OrbitColoringSolution`

```
@dataclass(frozen=True)
class OrbitColoringSolution:
    subgroup: Subgroup
    orbit_states: tuple[str, ...]
    occupation: SiteOccupation
```

`orbit_states` follows the deterministic orbit order produced by MS1. The expanded site occupation is derived from these orbit labels.

5.3 Verified solution

```
@dataclass(frozen=True)
class OccupationSymmetrySolution:
    occupation: SiteOccupation
    canonical_signature: tuple[str, ...]
    construction_subgroup: Subgroup
    occupation_stabilizer: Subgroup
    stabilizer_generators: tuple[int, ...]
    inequivalent_occupied_orbit_count: int
    subgroup_conjugacy_class_size: int
    orbit_size_under_parent: int
    canonicalizing_operation: int
```

The stored occupation is the canonical parent-orbit representative. `construction_subgroup` is conjugated by the same canonicalizing operation, so it remains a subgroup of the verified stabilizer.

6. Canonicalization and direct stabilizer verification

6.1 Signature transformation

For an old-site-to-new-site permutation p_g, the transformed signature is

$$(g \cdot c)_{p_g(i)} = c_i.$$

`transform_occupation_signature` implements this convention and requires a bijection.

6.2 Canonical signature

For every parent operation, form $g \cdot c$. The canonical representative is

$$\mathrm{can}(c) = \min_{g \in G} g \cdot c,$$

using the canonical full state rank defined in Section 4.1:

$$\emptyset < \mathrm{H} < \mathrm{Li} < \mathrm{Na} < \mathrm{K} < \mathrm{Rb} < \mathrm{Cs} < \mathrm{Fr} < \mathrm{Be} < \mathrm{Mg} < \mathrm{Ca} < \mathrm{Sr} < \mathrm{Ba} < \mathrm{Ra}.$$

When multiple operations give the same canonical signature, the smallest operation index is selected. `canonicalize_occupation_signature` returns both the signature and this operation.

6.3 Actual stabilizer

Every constructed candidate is rescored directly:

$$H_{\mathrm{occ}}(c) = \{g \in G : g \cdot c = c\}.$$

The fixed-operation set is converted back to a validated `Subgroup`. The reported symmetry is never inferred solely from the subgroup used to generate the coloring.

6.4 Orbit-stabilizer invariant

For every verified candidate,

$$|\operatorname{Orb}_G(c)|\,|H_{\mathrm{occ}}(c)| = |G|.$$

MS2 evaluates the parent orbit explicitly and raises `OccupationOptimizationError` if this identity fails.

7. Exact orbit-coloring solver

7.1 Deterministic orbit order

Subgroup orbits are ordered by:

1. ring-family order;
2. decreasing orbit size;
3. smallest site index.

Large orbits are assigned first to improve pruning. The order is independent of subgroup discovery history.

7.2 Dynamic-programming state

The feasibility state is

$$(i, \mathbf{n}),$$

where i is the next orbit and $\mathbf{n}$ is the flattened tuple of remaining counts for every ring family and state.

For the next orbit of size m in ring family r, branch over states α satisfying

$$n_{r,\alpha} \geq m,$$

then replace

$$n_{r,\alpha} \leftarrow n_{r,\alpha} - m.$$

Memoization stores exact feasibility of each distinct state. A solution is emitted only when all remaining counts are zero.

7.3 Fast rejection

Before recursion, MS2 applies necessary checks for each ring family:

- orbit capacities sum to the physical-site capacity;
- each requested state count is divisible by the greatest common divisor of the orbit sizes;
- each requested state count is individually representable by a subset of orbit sizes.

During recursion, remaining state counts must sum exactly to the remaining orbit capacity in every ring family.

These tests reject impossible subgroups without changing completeness.

7.4 Search-node guard

A coloring-search node is one distinct memoized state $(i, \mathbf{n})$. A shared budget applies across all tested subgroup classes.

Exceeding `max_coloring_search_nodes` raises

`OccupationColoringSearchIncompleteError`

and returns no partial optimum or partial tie set.

8. Maximum-order subgroup search

8.1 Search domain

MS2 uses the exhaustive `SubgroupLattice` from MS1. If one is not supplied, it calls

```
enumerate_subgroup_lattice(
    group,
    max_search_nodes=max_subgroup_search_nodes,
)
```

A supplied lattice must have identical multiplication and inverse tables, the same identity, and an exhaustive report.

Occupation feasibility is invariant under full-parent conjugacy because the physical-site domain and ring-family counts are parent-invariant. Therefore the admissible conjugating group is

$$C = G_{\text{site}}.$$

Only one representative of each G_{site} subgroup conjugacy class is tested.

8.2 Descending-order proof

Subgroup classes are tested in descending order. The first feasible order q satisfies

$$q = \max_{c} |H_{\text{occ}}(c)|.$$

Proof: for every occupation c, its stabilizer $H_{\text{occ}}(c)$ is a subgroup of G. The exhaustive search contains this subgroup or a parent-conjugate representative. A feasible occupation invariant under it exists by definition. Therefore no occupation can have stabilizer order larger than the first feasible subgroup order.

Every larger order must be exhausted before MS2 reports an optimum.

8.3 Actual stabilizer consistency

For a candidate generated from subgroup H,

$$H \leq H_{\text{occ}}(c).$$

At the first feasible order, direct verification must give

$$|H_{\text{occ}}(c)| = |H|.$$

A larger verified stabilizer would contradict the already exhausted larger-order search and is treated as an internal consistency failure.

9. Tie enumeration and ranking

9.1 `first` mode

`max_types="first"`

returns one deterministic canonical occupation after the optimum order is proven. Equal-order subgroup classes and colorings need not be exhausted. The certificate therefore marks tie enumeration as intentionally truncated.

9.2 `all` mode

`max_types="all"`

enumerates parent-symmetry-inequivalent optimum occupations. Canonical signatures deduplicate candidates under the complete parent action.

Enumeration stops after discovering one more unique occupation than `max_results`. The extra result proves truncation; only the first `max_results` ranked occupations are retained.

If no extra result is found and all optimum-order subgroup classes and colorings are exhausted, the certificate marks optimum tie enumeration complete.

9.3 Deterministic tie ordering

All returned MS2 candidates have the same certified maximum stabilizer order.

When `all` exhausts every optimum tie, the complete set is ordered by:

1. fewer inequivalent occupied cation orbits under the verified stabilizer;
2. canonical occupation signature;
3. stabilizer operation indices.

When `first` is requested, or when bounded `all` enumeration is truncated, MS2 guarantees deterministic output but does not claim a globally optimal secondary ranking among unenumerated ties. Density scoring, pair-distance filtering, and final physical ranking are deferred to MS4.

10. Optimality certificate

```python
@dataclass(frozen=True)
class OccupationOptimalityCertificate:
    parent_group_order: int
    admissible_conjugating_group_order: int
    subgroup_count: int
    subgroup_search_nodes: int
    max_subgroup_search_nodes: int
    conjugacy_orbits_tested: int
    represented_subgroups_tested: int
    exhausted_larger_orders: tuple[int, ...]
    optimum_order: int
    larger_orders_exhausted: bool
    optimality_proven: bool
    type_enumeration_mode: str
    optimum_order_conjugacy_classes_exhaustive: bool
    tie_enumeration_truncated: bool
    returned_unique_occupation_count: int
    coloring_search_nodes: int
    max_coloring_search_nodes: int
```

The certificate is valid only when:

- subgroup enumeration is exhaustive;

- every subgroup conjugacy class above the optimum is infeasible;
- no coloring-search budget is exceeded;
- direct stabilizer and orbit-stabilizer checks pass.

`highest_exhausted_order` is a derived property. `exhausted_larger_orders` is the complete descending tuple of subgroup orders above the certified optimum.

11. Main public API

11.1 Constraint resolution

```
resolve_population_constraints(
    site_ring_classes,
    *,
    counts_by_ring=None,
    ratios_by_ring=None,
) -> ResolvedPopulationConstraints
```

11.2 Orbit coloring

```
iter_orbit_colorings(
    *,
    group,
    subgroup,
    physical_site_action,
    site_ids,
    site_ring_classes,
    constraints,
    budget,
) -> Iterator[OrbitColoringSolution]
```

`OccupationColoringBudget` is a small mutable search counter shared across subgroup tests. The iterator is public primarily for focused testing and MS3 reuse; normal callers should use the optimizer.

11.3 Maximum-symmetry occupation optimizer

```
optimize_maximum_symmetry_occupations(
    template_model,
    symmetry_model,
    *,
    counts_by_ring=None,
    ratios_by_ring=None,
    resolved_constraints=None,
    subgroup_lattice=None,
    max_subgroup_search_nodes=1_000_000,
    max_coloring_search_nodes=1_000_000,
    max_results=1,
    max_types="first",
) -> MaximumSymmetryOccupationResult
```

Exactly one of the following input forms is used:

- `counts_by_ring`;
- `ratios_by_ring`;
- precomputed `resolved_constraints`.

`max_types` accepts `first` or `all`.

12. Failure semantics

`PopulationConstraintError` Invalid counts, ratios, states, capacities, ring labels, or site ordering.

`OccupationColoringSearchIncompleteError` The exact dynamic-programming state budget was exceeded. No partial result is returned.

`NoFeasibleOccupationError` No subgroup coloring satisfies the resolved populations. With a valid trivial subgroup and valid capacities this normally indicates an inconsistent external model.

`OccupationOptimizationError` Algebraic invariant failure, invalid certificate, nonexhaustive supplied lattice, orbit-stabilizer failure, or disagreement between construction and verified stabilizers.

MS1's `MaximumSymmetrySearchIncompleteError` propagates unchanged when exhaustive subgroup enumeration exceeds its budget.

13. Complexity

Let:

- $|G|$ be the parent-group order;
- S be the number of subgroups;
- m_H be the number of physical-site orbits under subgroup H;
- $N_{r,\alpha}$ be exact state counts.

The subgroup lattice is constructed in MS1. MS2 tests one representative per parent-conjugacy class.

For one subgroup, the memoized orbit-coloring state space is bounded by

$$(m_H + 1) \prod_{r,\alpha} (N_{r,\alpha} + 1),$$

but ring-capacity conservation, orbit-size divisibility, subset-sum rejection, and descending orbit sizes reduce the reachable set substantially.

Direct canonicalization and stabilizer verification cost

$$O(|G|\,|\mathcal{P}|)$$

per emitted candidate.

The implementation is intended for the 46 grouped sites of primitive LTA and small fixed-cell crystallographic groups. It is not a general large-supercell configurational enumerator.

14. Verification requirements

Focused tests must cover:

1. exact count and ratio resolution;
2. inferred vacancies and explicit-vacancy consistency;
3. capacity and state validation;
4. full-group symmetry for uniform occupation;
5. site stabilizer for one occupied site;
6. alternating half-filled C_4 occupation with order-two stabilizer;
7. order-four optimum for a two-site square occupation under D_4;
8. equal-order nonconjugate optimum types under the Klein four group;
9. deterministic bounded all-ties truncation;
10. first mode proving the optimum before returning;
11. hard coloring-budget failure;
12. explicit canonical state order;
13. species sensitivity;

14. multiple ring-family constraints;
15. brute-force oracle agreement for small groups;
16. synthetic crystallographic integration through MS0-MS2;
17. reuse of an externally enumerated exhaustive subgroup lattice;
18. rejection of a noncompositional physical-site action;
19. regression of all MS0, MS1, random-population, and unique-batch tests.

For every small fixture,

$$|\operatorname{Orb}_G(c)|\,|H_{\mathrm{occ}}(c)| = |G|$$

must hold.

15. MS3 handoff

MS3 will reuse:

- `ResolvedPopulationConstraints`;
- subgroup traversal and orbit partitions;
- the orbit-coloring dynamic-programming structure;
- `SiteOccupation`;
- canonicalization and stabilizer utilities;
- search budgets and certificate patterns.

MS3 extends subgroup feasibility by requiring a symmetry-consistent positional variant for every occupied physical-site orbit. MS2 occupation results are therefore exact upper bounds for coordinate symmetry, but they are not yet atomically realized structures.

18. MS7 hybrid occupation search

The optimizer computes the exact raw occupation count and may use direct exhaustive enumeration when it is no larger than `max_direct_occupation_count`. Direct and subgroup-orbit paths share canonicalization, stabilizer reconstruction, and optimality-certificate validation. The certificate reports the selected strategy and raw count.